

Basic Banking API with Python and FastAPI

Step-by-step build guide for students

What you will build	A REST API where users register, log in, view their account, deposit, withdraw, transfer money, see their transaction history and log out.
Stack	Python 3.11+, FastAPI, JWT, SQLite, SQLAlchemy, Pydantic, pytest
You will finish with	A public GitHub repository with source code, tests, README and Swagger docs
How long	About 4 to 6 hours if you follow every step and read the explanations

HOW TO USE THIS GUIDE

1. Work through the steps **in order**. Each step builds on the one before it.
2. **Type every file yourself**. Do not copy-paste. Typing slowly is how you learn what each line does, and you will be asked to explain your code.
3. Dark boxes are **commands** for your terminal. Light boxes labelled FILE are **code** for your editor.
4. After each step there is a **checkpoint**. If it fails, fix it before continuing. Small bugs are easy to find; bugs found 10 steps later are not.
5. Commit to Git after every few steps. Good commit history shows good habits.

Commands are shown for **macOS / Linux** first, then **Windows (PowerShell)** where they differ.

The big picture

The 8 endpoints you will build

#	Method	Path	Login needed?	Purpose
1	POST	/api/auth/register	No	Create a user and their bank account
2	POST	/api/auth/login	No	Get a JWT access token
3	GET	/api/accounts/me	Yes	Your account details
4	POST	/api/accounts/deposit	Yes	Add money
5	POST	/api/accounts/withdraw	Yes	Take money out
6	POST	/api/accounts/transfer	Yes	Send money to another account
7	GET	/api/accounts/transactions	Yes	Your transaction history
8	POST	/api/auth/logout	Yes	Invalidate your token

How the code is organised (layers)

A request travels down through three layers and the answer travels back up. Each layer has **one job**. This is what 'clean project structure' means in the assessment.

Layer	Folder	Its one job	It must NOT
API (routes)	app/api	Receive the HTTP request, call a service, return the response	Contain business rules or SQL
Services	app/services	Business rules: 'can this user withdraw this much?' and transaction handling	Know about HTTP
Repositories	app/repositories	Database queries only (select, insert)	Decide business rules
Models	app/models	Describe database tables	
Schemas	app/schemas	Describe and validate the JSON going in and out (Pydantic)	
Core / DB	app/core, app/db	Settings, password hashing, JWT, errors, database connection	

Final folder structure

```
bank-api/  
  app/  
    main.py  
    api/      deps.py  auth.py  accounts.py  
    core/     config.py security.py exceptions.py  
    db/       base.py  session.py  
    models/   user.py  account.py transaction.py blacklisted_token.py  
    schemas/  base.py  auth.py  account.py transaction.py  
    services/ auth_service.py account_service.py  
    repositories/ user_repository.py account_repository.py  
                  transaction_repository.py token_repository.py  
    tests/     conftest.py test_auth.py test_accounts.py  
  requirements.txt  
  README.md  
  .env.example  
  .gitignore
```

Every folder inside app/ also contains an empty file called `__init__.py`. That empty file tells Python 'this folder is a package you can import from'.

Step 1: Check your tools

Goal: Make sure Python and Git are installed.

macOS / Linux

```
python3 --version
git --version
```

Windows (PowerShell)

```
python --version
git --version
```

You need **Python 3.11 or newer**. If a command is not found, install Python from python.org (on Windows tick 'Add Python to PATH') and Git from git-scm.com, then open a new terminal and try again.

CHECKPOINT - do not move on until this works

Both commands print a version number. Python shows 3.11 or higher.

Step 2: Create the project folder and a virtual environment

Goal: Give the project its own private Python so its libraries do not clash with other projects.

WHY WE DO THIS

A **virtual environment** (venv) is a folder that holds this project's libraries only. Anyone who clones your repository can rebuild it from requirements.txt. The venv folder itself is **never** committed to Git.

macOS / Linux

```
mkdir bank-api
cd bank-api
python3 -m venv venv
source venv/bin/activate
```

Windows (PowerShell)

```
mkdir bank-api
cd bank-api
python -m venv venv
venv\Scripts\activate
```

If PowerShell blocks activation, run this once and try again: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`

CHECKPOINT - do not move on until this works

Your terminal prompt now starts with (venv). **Every time you open a new terminal you must activate the venv again.**

Optional: open the folder in VS Code

```
code .
```

Step 3: Install the libraries

Goal: Create requirements.txt and install everything the project needs.

Create a file called requirements.txt in the project root and type this:

FILE: requirements.txt

```
fastapi==0.141.1
uvicorn[standard]==0.54.0
SQLAlchemy==2.1.1
pydantic[email]==2.13.5
pydantic-settings==2.15.0
PyJWT==2.15.1
bcrypt==5.0.0
pytest==9.1.1
httpx==0.28.1
```

```
pip install --upgrade pip
pip install -r requirements.txt
```

Library	What we use it for
fastapi	The web framework. Turns Python functions into API endpoints and builds the Swagger docs
uvicorn	The server that actually runs a FastAPI app
SQLAlchemy	ORM: lets us work with database tables as Python classes
pydantic[email]	Validates request/response data. The [email] extra checks that an email is valid
pydantic-settings	Reads settings from a .env file
PyJWT	Creates and verifies JWT tokens
bcrypt	Hashes passwords safely
pytest, httpx	Running tests and making test requests to the API

CHECKPOINT - do not move on until this works

Run `pip list` and confirm fastapi, sqlalchemy, bcrypt and pytest appear.

Step 4: Create the folders and __init__.py files

Goal: Build the empty skeleton of the project.

macOS / Linux - folders

```
mkdir -p app/api app/core app/db app/models
mkdir -p app/schemas app/services app/repositories tests
```

macOS / Linux - empty files

```
touch app/__init__.py app/api/__init__.py app/core/__init__.py app/db/__init__.py
touch app/models/__init__.py app/schemas/__init__.py app/services/__init__.py
touch app/repositories/__init__.py tests/__init__.py
```

Windows (PowerShell) - folders

```
mkdir app\api, app\core, app\db, app\models
mkdir app\schemas, app\services, app\repositories, tests
```

Windows (PowerShell) - empty files

```
New-Item app\__init__.py, app\api\__init__.py -ItemType File
New-Item app\core\__init__.py, app\db\__init__.py -ItemType File
New-Item app\models\__init__.py, app\schemas\__init__.py -ItemType File
New-Item app\services\__init__.py, app\repositories\__init__.py -ItemType File
New-Item tests\__init__.py -ItemType File
```

CHECKPOINT - do not move on until this works

Run `ls app` (Windows: `dir app`) and you can see `api`, `core`, `db`, `models`, `schemas`, `services`, `repositories`.

Step 5: Git, .gitignore and environment files

Goal: Start version control and make sure secrets never reach GitHub.

```
git init
```

Create `.gitignore` **before your first commit**:

FILE: .gitignore

```
venv/
.venv/
__pycache__/
*.pyc
.env
*.db
.pytest_cache/
.idea/
.vscode/
```

Create `.env.example`. This is a **template** with no real secret. It is committed to Git:

FILE: .env.example

```
DATABASE_URL=sqlite:///./bank.db
SECRET_KEY=replace-this-with-a-long-random-string
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=30
```

Now create your real `.env` by copying the template:

macOS / Linux

```
cp .env.example .env
```

Windows (PowerShell)

```
copy .env.example .env
```

Generate a random secret and paste it as the value of `SECRET_KEY` inside `.env`:

```
python -c "import secrets; print(secrets.token_hex(32))"
```

WHY WE DO THIS

The **SECRET_KEY** signs every JWT. If someone learns it they can forge tokens and log in as anyone. That is why `.env` (real secret) is ignored by Git and only `.env.example` (fake placeholder) is committed. This is an explicit requirement of the assessment.

CHECKPOINT - do not move on until this works

Run `git status`. You should see `.env.example` and `.gitignore` but **not** `.env` and **not** the `venv` folder.

```
git add .
git commit -m "Project skeleton"
```

Step 6: Configuration: app/core/config.py

Goal: Load settings from the .env file in one central place.

FILE: app/core/config.py

```
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    database_url: str = "sqlite:///./bank.db"
    secret_key: str
    algorithm: str = "HS256"
    access_token_expire_minutes: int = 30

    model_config = SettingsConfigDict(env_file=".env", extra="ignore")

settings = Settings()
```

WHY WE DO THIS

BaseSettings reads environment variables and the .env file and converts them to the right types. `secret_key: str` has **no default**, so the app refuses to start if you forget to set it. That is safer than shipping with a default secret. `extra="ignore"` stops errors if .env has variables we do not declare.

Start Python, then type each line after the `>>>` prompt (do not type the `>>>` itself), one at a time

```
python
>>> from app.core.config import settings
>>> print(settings.database_url, settings.algorithm)
>>> exit()
```

CHECKPOINT - do not move on until this works

It prints `sqlite:///./bank.db HS256`. If you see a 'secret_key field required' error, your .env file is missing or SECRET_KEY is empty.

Step 7: Custom errors: app/core/exceptions.py

Goal: Create our own error classes so business code can say what went wrong without knowing about HTTP.

FILE: app/core/exceptions.py

```
class AppError(Exception):
    """Base class for errors we raise on purpose."""

    def __init__(self, status_code: int, detail: str):
        self.status_code = status_code
        self.detail = detail
        super().__init__(detail)

class BadRequestError(AppError):
    def __init__(self, detail: str = "Bad request"):
        super().__init__(400, detail)

class UnauthorizedError(AppError):
    def __init__(self, detail: str = "Not authenticated"):
        super().__init__(401, detail)

class NotFoundError(AppError):
    def __init__(self, detail: str = "Not found"):
        super().__init__(404, detail)

class ConflictError(AppError):
    def __init__(self, detail: str = "Conflict"):
        super().__init__(409, detail)
```

WHY WE DO THIS

A service will write raise `BadRequestError('Insufficient balance')`. Later in `main.py` we teach FastAPI to turn any `AppError` into a JSON response with the right status code. The 'what went wrong' logic stays in the service; the 'how to answer over HTTP' logic stays in one place.

Class	HTTP status	Used when
<code>BadRequestError</code>	400	Insufficient balance, transfer to yourself
<code>UnauthorizedError</code>	401	Missing/invalid/expired/revoked token, wrong login
<code>NotFoundError</code>	404	Receiver account does not exist
<code>ConflictError</code>	409	Email already registered

Step 8: Database connection: app/db/base.py and app/db/session.py

Goal: Connect to the database and give each request its own session.

FILE: app/db/base.py

```
from datetime import datetime, timezone

from sqlalchemy.orm import DeclarativeBase

class Base(DeclarativeBase):
    pass

def utcnow() -> datetime:
    return datetime.now(timezone.utc)
```

FILE: app/db/session.py

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

from app.core.config import settings

connect_args = {}
if settings.database_url.startswith("sqlite"):
    connect_args = {"check_same_thread": False}

engine = create_engine(settings.database_url, connect_args=connect_args)
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False)

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

WHY WE DO THIS

Base is the parent class of all our table classes. **engine** is the connection to the database. A **session** is a short conversation with the database: you add/change objects and then *commit* or *rollback*. `get_db` opens one session per request and always closes it (the `finally` block). FastAPI will call it for us using dependency injection later. `check_same_thread=False` is only needed for SQLite because FastAPI uses several threads.

Changing database later: to use PostgreSQL you would only change `DATABASE_URL` in `.env` (for example `postgresql://user:pass@localhost/bank`) and install the driver. No code changes.

Step 9: Database tables (models)

Goal: Describe the four tables as Python classes.

FILE: app/models/user.py

```
from datetime import datetime

from sqlalchemy import DateTime, String
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.db.base import Base, utcnow

class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(100))
    email: Mapped[str] = mapped_column(String(255), unique=True, index=True)
    password_hash: Mapped[str] = mapped_column(String(255))
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)

    account: Mapped["Account"] = relationship(back_populates="user", uselist=False)
```


FILE: app/models/account.py

```
from datetime import datetime
from decimal import Decimal

from sqlalchemy import CheckConstraint, DateTime, ForeignKey, Numeric, String
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.db.base import Base, utcnow

class Account(Base):
    __tablename__ = "accounts"
    __table_args__ = (
        CheckConstraint("balance >= 0", name="ck_accounts_balance_non_negative"),
    )

    id: Mapped[int] = mapped_column(primary_key=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), unique=True)
    account_number: Mapped[str] = mapped_column(String(20), unique=True, index=True)
    balance: Mapped[Decimal] = mapped_column(Numeric(12, 2), default=Decimal("0.00"))
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)

    user: Mapped["User"] = relationship(back_populates="account")
```

FILE: app/models/transaction.py

```
import enum
from datetime import datetime
from decimal import Decimal

from sqlalchemy import DateTime, ForeignKey, Numeric, String
from sqlalchemy.orm import Mapped, mapped_column

from app.db.base import Base, utcnow

class TransactionType(str, enum.Enum):
    DEPOSIT = "DEPOSIT"
    WITHDRAW = "WITHDRAW"
    TRANSFER_DEBIT = "TRANSFER_DEBIT"
    TRANSFER_CREDIT = "TRANSFER_CREDIT"

class TransactionStatus(str, enum.Enum):
    SUCCESS = "SUCCESS"
    FAILED = "FAILED"

class Transaction(Base):
    __tablename__ = "transactions"

    id: Mapped[int] = mapped_column(primary_key=True)
    account_id: Mapped[int] = mapped_column(ForeignKey("accounts.id"), index=True)
    type: Mapped[str] = mapped_column(String(20))
    amount: Mapped[Decimal] = mapped_column(Numeric(12, 2))
    status: Mapped[str] = mapped_column(String(20), default=TransactionStatus.SUCCESS.value)
    reference_id: Mapped[str | None] = mapped_column(String(36), nullable=True)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)
```

FILE: app/models/blacklisted_token.py

```
from datetime import datetime

from sqlalchemy import DateTime, String
from sqlalchemy.orm import Mapped, mapped_column

from app.db.base import Base

class BlacklistedToken(Base):
    __tablename__ = "blacklisted_tokens"

    id: Mapped[int] = mapped_column(primary_key=True)
    jti: Mapped[str] = mapped_column(String(64), unique=True, index=True)
    expires_at: Mapped[datetime] = mapped_column(DateTime(timezone=True))
```

FILE: app/models/__init__.py

```
from app.models.account import Account
from app.models.blacklisted_token import BlacklistedToken
from app.models.transaction import Transaction, TransactionStatus, TransactionType
from app.models.user import User

__all__ = [
    "Account",
    "BlacklistedToken",
    "Transaction",
    "TransactionStatus",
    "TransactionType",
    "User",
]
```

WHY WE DO THIS

Numeric(12, 2) stores money exactly (up to 12 digits, 2 after the decimal point). Never use float for money: in float, $0.1 + 0.2$ is 0.30000000000000004 .

CheckConstraint('balance >= 0') is a last safety net: even if our code had a bug, the database itself refuses a negative balance.

unique=True on email means the database rejects duplicates even if two people register at the exact same moment. On `accounts.user_id` it means one account per user.

Transaction.type uses an Enum so only valid values exist. A transfer creates **two** rows (TRANSFER_DEBIT for the sender, TRANSFER_CREDIT for the receiver) sharing one `reference_id`, so each user only sees their own side.

BlacklistedToken stores revoked token ids for logout (Step 13 explains this).

Start Python, then type each line after the `>>>` prompt (do not type the `>>>` itself), one at a time

```
python
>>> from app import models
>>> from app.db.base import Base
>>> print(sorted(Base.metadata.tables))
>>> exit()
```

CHECKPOINT - do not move on until this works

It prints `['accounts', 'blacklisted_tokens', 'transactions', 'users']`.

```
git add .
git commit -m "Config, errors, database and models"
```

Step 10: Passwords and tokens: app/core/security.py

Goal: Hash passwords and create/verify JWTs.

FILE: app/core/security.py

```
import uuid
from datetime import datetime, timedelta, timezone

import bcrypt
import jwt

from app.core.config import settings

def hash_password(password: str) -> str:
    return bcrypt.hashpw(password.encode("utf-8"), bcrypt.gensalt()).decode("utf-8")

def verify_password(password: str, password_hash: str) -> bool:
    return bcrypt.checkpw(password.encode("utf-8"), password_hash.encode("utf-8"))

def create_access_token(user_id: int) -> str:
    now = datetime.now(timezone.utc)
    payload = {
        "sub": str(user_id),
        "jti": uuid.uuid4().hex,
        "iat": now,
        "exp": now + timedelta(minutes=settings.access_token_expire_minutes),
    }
    return jwt.encode(payload, settings.secret_key, algorithm=settings.algorithm)

def decode_access_token(token: str) -> dict:
    """Raises jwt.ExpiredSignatureError or jwt.InvalidTokenError."""
    return jwt.decode(token, settings.secret_key, algorithms=[settings.algorithm])
```

WHY WE DO THIS

Hashing is one-way: we store a scrambled version of the password. Even if the database leaks, nobody can read the passwords. **bcrypt.gensalt()** adds random salt so two users with the same password get different hashes. We can only check a login by hashing the typed password and comparing (checkpw).

A **JWT** is a signed piece of text that says 'this is user 5, valid until 10:30'. The server does not store it; it just verifies the signature. Ours contains: **sub** (user id), **jti** (unique token id, needed for logout), **iat** (issued at) and **exp** (expiry). PyJWT automatically rejects tokens whose exp has passed.

Start Python, then type each line after the >>> prompt (do not type the >>> itself), one at a time

```
python
>>> from app.core.security import *
>>> h = hash_password('Password123')
>>> print(h)
>>> print(verify_password('Password123', h), verify_password('wrong', h))
>>> t = create_access_token(1)
>>> print(t)
>>> print(decode_access_token(t))
>>> exit()
```

CHECKPOINT - do not move on until this works

The hash starts with \$2b\$, the next line prints True False, then you see a long token and a dictionary with sub, jti, iat and exp.

Step 11: Request/response shapes: schemas

Goal: Use Pydantic to validate everything coming in and control everything going out.

FILE: app/schemas/base.py

```
from pydantic import BaseModel, ConfigDict
from pydantic.alias_generators import to_camel

class CamelModel(BaseModel):
    """Accepts and returns camelCase JSON, uses snake_case in Python."""

    model_config = ConfigDict(
        alias_generator=to_camel,
        populate_by_name=True,
        from_attributes=True,
    )
```

FILE: app/schemas/auth.py

```
from datetime import datetime
from typing import Annotated

from pydantic import EmailStr, StringConstraints, field_validator

from app.schemas.base import CamelModel

Name = Annotated[str, StringConstraints(strip_whitespace=True, min_length=1, max_length=100)]

class RegisterRequest(CamelModel):
    name: Name
    email: EmailStr
    password: str

    @field_validator("email")
    @classmethod
    def normalise_email(cls, value: str) -> str:
        return value.lower()

    @field_validator("password")
    @classmethod
    def check_password_strength(cls, value: str) -> str:
        if len(value) < 8:
            raise ValueError("Password must be at least 8 characters long")
        if len(value.encode("utf-8")) > 72:
            raise ValueError("Password must be at most 72 bytes long")
        if not any(c.islower() for c in value):
            raise ValueError("Password must contain a lowercase letter")
        if not any(c.isupper() for c in value):
            raise ValueError("Password must contain an uppercase letter")
        if not any(c.isdigit() for c in value):
            raise ValueError("Password must contain a digit")
        return value

class LoginRequest(CamelModel):
    email: EmailStr
    password: str

    @field_validator("email")
    @classmethod
    def normalise_email(cls, value: str) -> str:
        return value.lower()

class RegisterResponse(CamelModel):
    id: int
    name: str
    email: str
    account_number: str
```

```

        created_at: datetime

class TokenResponse(CamelModel):
    access_token: str
    token_type: str = "bearer"

class MessageResponse(CamelModel):
    message: str

```

FILE: app/schemas/account.py

```

from datetime import datetime
from decimal import Decimal
from typing import Annotated

from pydantic import Field

from app.schemas.base import CamelModel

Amount = Annotated[Decimal, Field(gt=0, max_digits=12, decimal_places=2)]

class AmountRequest(CamelModel):
    amount: Amount

class TransferRequest(CamelModel):
    receiver_account_id: Annotated[int, Field(gt=0)]
    amount: Amount

class AccountResponse(CamelModel):
    account_id: int
    account_number: str
    account_holder_name: str
    balance: float
    created_at: datetime

```

FILE: app/schemas/transaction.py

```

from datetime import datetime

from app.schemas.base import CamelModel

class TransactionResponse(CamelModel):
    id: int
    type: str
    amount: float
    status: str
    reference_id: str | None = None
    created_at: datetime

class OperationResponse(CamelModel):
    message: str
    balance: float
    transaction: TransactionResponse

```

WHY WE DO THIS

Pydantic models are the **bouncer at the door**. Bad input (bad email, weak password, amount 0, negative amount, blank name) is rejected automatically with status **422** before it reaches your code. This satisfies the Validation section of the assessment.

CamelModel: the assessment's JSON uses camelCase (accessToken, receiverAccountId) while Python style is snake_case. The alias_generator converts automatically, so we write access_token in Python and clients see accessToken.

Response models never contain password_hash. Whatever is not declared in a response model can never leak.

Amount is a Decimal with gt=0 and max 2 decimal places. That rejects 0, negatives, text and 10.999.

Start Python, then type each line after the >>> prompt (do not type the >>> itself), one at a time

```
python
>>> from app.schemas.auth import RegisterRequest as R
>>> R(name='A', email='bad', password='x')
>>> print(R(name=' Ann ', email='ANN@X.com', password='Password1'))
>>> exit()
```

CHECKPOINT - do not move on until this works

The first R(...) call fails with a long ValidationError listing the invalid email and the password rules - exactly what we want. The print line succeeds and shows name='Ann' (spaces trimmed) and a lowercase email.

Step 12: Repositories: database queries

Goal: Put every database query in one layer so the rest of the code never writes SQL.

FILE: app/repositories/user_repository.py

```
from sqlalchemy import select
from sqlalchemy.orm import Session

from app.models import User

class UserRepository:
    def __init__(self, db: Session):
        self.db = db

    def get_by_id(self, user_id: int) -> User | None:
        return self.db.get(User, user_id)

    def get_by_email(self, email: str) -> User | None:
        return self.db.scalar(select(User).where(User.email == email))

    def create(self, name: str, email: str, password_hash: str) -> User:
        user = User(name=name, email=email, password_hash=password_hash)
        self.db.add(user)
        self.db.flush()
        return user
```

FILE: app/repositories/account_repository.py

```
from sqlalchemy import select
from sqlalchemy.orm import Session

from app.models import Account

class AccountRepository:
    def __init__(self, db: Session):
        self.db = db

    def get_by_id(self, account_id: int) -> Account | None:
        return self.db.get(Account, account_id)

    def get_by_user_id(self, user_id: int) -> Account | None:
        return self.db.scalar(select(Account).where(Account.user_id == user_id))

    def get_for_update(self, account_id: int) -> Account | None:
        """Loads the row and locks it until commit or rollback."""
        stmt = select(Account).where(Account.id == account_id).with_for_update()
        return self.db.scalar(stmt)

    def account_number_exists(self, account_number: str) -> bool:
        stmt = select(Account.id).where(Account.account_number == account_number)
        return self.db.scalar(stmt) is not None

    def create(self, user_id: int, account_number: str) -> Account:
        account = Account(user_id=user_id, account_number=account_number)
        self.db.add(account)
        self.db.flush()
        return account
```

FILE: app/repositories/transaction_repository.py

```
from decimal import Decimal

from sqlalchemy import select
from sqlalchemy.orm import Session

from app.models import Transaction, TransactionStatus

class TransactionRepository:
    def __init__(self, db: Session):
        self.db = db

    def create(
        self,
        account_id: int,
        type_: str,
        amount: Decimal,
        reference_id: str | None = None,
    ) -> Transaction:
        transaction = Transaction(
            account_id=account_id,
            type=type_,
            amount=amount,
            status=TransactionStatus.SUCCESS.value,
            reference_id=reference_id,
        )
        self.db.add(transaction)
        self.db.flush()
        return transaction

    def list_for_account(self, account_id: int) -> list[Transaction]:
        stmt = (
            select(Transaction)
            .where(Transaction.account_id == account_id)
            .order_by(Transaction.id)
        )
        return list(self.db.scalars(stmt))
```

FILE: app/repositories/token_repository.py

```
from datetime import datetime, timezone

from sqlalchemy import delete, select
from sqlalchemy.orm import Session

from app.models import BlacklistedToken

class TokenRepository:
    def __init__(self, db: Session):
        self.db = db

    def add(self, jti: str, expires_at: datetime) -> None:
        self.db.add(BlacklistedToken(jti=jti, expires_at=expires_at))
        self.db.flush()

    def is_blacklisted(self, jti: str) -> bool:
        stmt = select(BlacklistedToken.id).where(BlacklistedToken.jti == jti)
        return self.db.scalar(stmt) is not None

    def delete_expired(self) -> None:
        """Expired tokens are rejected anyway, so their rows can go."""
        self.db.execute(
            delete(BlacklistedToken).where(
                BlacklistedToken.expires_at < datetime.now(timezone.utc)
            )
        )
```

WHY WE DO THIS

Repositories only **read and write**; they never decide rules and never raise HTTP errors. `db.flush()` sends the INSERT to the database so the new row gets its id, but does **not** finish the transaction. The service decides when to commit().

get_for_update uses `.with_for_update()` which locks the account row until commit or rollback. If two requests try to change the same balance at once, the second waits its turn. This is how we handle the 'concurrent transactions' edge case.

Step 13: Services: the business rules

Goal: Write the rules of banking, including the atomic transfer.

FILE: app/services/auth_service.py

```
import secrets
from datetime import datetime, timezone

from sqlalchemy.exc import IntegrityError
from sqlalchemy.orm import Session

from app.core.exceptions import ConflictError, UnauthorizedError
from app.core.security import create_access_token, hash_password, verify_password
from app.models import Account, User
from app.repositories.account_repository import AccountRepository
from app.repositories.token_repository import TokenRepository
from app.repositories.user_repository import UserRepository
from app.schemas.auth import LoginRequest, RegisterRequest

class AuthService:
    def __init__(self, db: Session):
        self.db = db
        self.users = UserRepository(db)
        self.accounts = AccountRepository(db)
        self.tokens = TokenRepository(db)

    def _generate_account_number(self) -> str:
        while True:
            number = "".join(str(secrets.randbelow(10)) for _ in range(10))
            if not self.accounts.account_number_exists(number):
                return number
```



```

def register(self, data: RegisterRequest) -> tuple[User, Account]:
    if self.users.get_by_email(data.email):
        raise ConflictError("Email is already registered")
    try:
        user = self.users.create(data.name, data.email, hash_password(data.password))
        account = self.accounts.create(user.id, self._generate_account_number())
        self.db.commit()
    except IntegrityError:
        self.db.rollback()
        raise ConflictError("Email is already registered")
    return user, account

def login(self, data: LoginRequest) -> str:
    user = self.users.get_by_email(data.email)
    if user is None or not verify_password(data.password, user.password_hash):
        raise UnauthorizedError("Invalid email or password")
    return create_access_token(user.id)

def logout(self, payload: dict) -> None:
    expires_at = datetime.fromtimestamp(payload["exp"], tz=timezone.utc)
    self.tokens.delete_expired()
    self.tokens.add(payload["jti"], expires_at)
    self.db.commit()

```

FILE: app/services/account_service.py

```

import uuid
from decimal import Decimal

from sqlalchemy.orm import Session

from app.core.exceptions import BadRequestError, NotFoundError
from app.models import Account, Transaction, TransactionType, User
from app.repositories.account_repository import AccountRepository
from app.repositories.transaction_repository import TransactionRepository

class AccountService:
    def __init__(self, db: Session):
        self.db = db
        self.accounts = AccountRepository(db)
        self.transactions = TransactionRepository(db)

    def get_account(self, user: User) -> Account:
        account = self.accounts.get_by_user_id(user.id)
        if account is None:
            raise NotFoundError("Account not found")
        return account

    def deposit(self, user: User, amount: Decimal) -> tuple[Account, Transaction]:
        own = self.get_account(user)
        try:
            account = self.accounts.get_for_update(own.id)
            account.balance += amount
            tx = self.transactions.create(account.id, TransactionType.DEPOSIT.value, amount)
            self.db.commit()
        except Exception:
            self.db.rollback()
            raise
        return account, tx

    def withdraw(self, user: User, amount: Decimal) -> tuple[Account, Transaction]:
        own = self.get_account(user)
        try:
            account = self.accounts.get_for_update(own.id)
            if account.balance < amount:
                raise BadRequestError("Insufficient balance")
            account.balance -= amount
            tx = self.transactions.create(account.id, TransactionType.WITHDRAW.value, amount)
            self.db.commit()
        except Exception:
            self.db.rollback()
            raise
        return account, tx

```

```

def transfer(
    self, user: User, receiver_account_id: int, amount: Decimal
) -> tuple[Account, Transaction]:
    sender_own = self.get_account(user)
    if receiver_account_id == sender_own.id:
        raise BadRequestError("Cannot transfer money to your own account")
    if self.accounts.get_by_id(receiver_account_id) is None:
        raise NotFoundError("Receiver account not found")

    try:
        # Lock both rows in a fixed order so two opposite transfers
        # can never wait on each other forever (deadlock).
        locked = {}
        for account_id in sorted([sender_own.id, receiver_account_id]):
            locked[account_id] = self.accounts.get_for_update(account_id)
        sender = locked[sender_own.id]
        receiver = locked[receiver_account_id]

        if sender.balance < amount:
            raise BadRequestError("Insufficient balance")

        sender.balance -= amount
        receiver.balance += amount

        reference = uuid.uuid4().hex
        debit = self.transactions.create(
            sender.id, TransactionType.TRANSFER_DEBIT.value, amount, reference
        )
        self.transactions.create(
            receiver.id, TransactionType.TRANSFER_CREDIT.value, amount, reference
        )
        self.db.commit() # one commit = all-or-nothing
    except Exception:
        self.db.rollback()
        raise
    return sender, debit

def history(self, user: User) -> list[Transaction]:
    account = self.get_account(user)
    return self.transactions.list_for_account(account.id)

```

WHY WE DO THIS

Registration: we check for a duplicate email first, but two people could register the same email at the same instant, so we also catch `IntegrityError` from the unique constraint and answer 409.

Login: unknown email and wrong password return the **same** message. Different messages would let an attacker discover which emails are registered.

Logout (token blacklist): a JWT cannot be 'deleted' because the server does not store it. So on logout we save its jti in the blacklist table, and every request checks that table. We delete rows whose token has expired anyway. (The README needs you to explain this strategy.)

ATOMIC TRANSFER - the most important part of the assessment

Atomic means all-or-nothing. In `transfer()`, five things must happen together: subtract from sender, add to receiver, write the sender's history row, write the receiver's history row. If step 4 crashes, steps 1-3 must vanish - otherwise money disappears.

How: we change everything in the session and call `commit()` **once at the end**. If *anything* raises an exception, the except block calls `rollback()`, which undoes all uncommitted changes, and re-raises the error.

Deadlock prevention: imagine Ann sends money to Bob while Bob sends to Ann. If each locked their own account first they would wait for each other forever. Locking the accounts **in ascending id order** means both requests lock the same account first, so one simply waits.

We proved it works with a test in Step 18 that forces a crash halfway through a transfer.

Step 14: Authentication dependencies: app/api/deps.py

Goal: Write the code that protects private endpoints.

FILE: app/api/deps.py

```
import jwt
from fastapi import Depends
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer
from sqlalchemy.orm import Session

from app.core.exceptions import UnauthorizedError
from app.core.security import decode_access_token
from app.db.session import get_db
from app.models import User
from app.repositories.token_repository import TokenRepository
from app.repositories.user_repository import UserRepository

# auto_error=False lets us return 401 (FastAPI's default would be 403)
bearer_scheme = HTTPBearer(auto_error=False)

def get_token_payload(
    credentials: HTTPAuthorizationCredentials | None = Depends(bearer_scheme),
    db: Session = Depends(get_db),
) -> dict:
    if credentials is None:
        raise UnauthorizedError("Not authenticated")
    try:
        payload = decode_access_token(credentials.credentials)
    except jwt.ExpiredSignatureError:
        raise UnauthorizedError("Token has expired")
    except jwt.InvalidTokenError:
        raise UnauthorizedError("Invalid token")

    if "jti" not in payload or "sub" not in payload:
        raise UnauthorizedError("Invalid token")
    if TokenRepository(db).is_blacklisted(payload["jti"]):
        raise UnauthorizedError("Token has been revoked")
    return payload

def get_current_user(
    payload: dict = Depends(get_token_payload),
    db: Session = Depends(get_db),
) -> User:
    user = UserRepository(db).get_by_id(int(payload["sub"]))
    if user is None:
        raise UnauthorizedError("Invalid token")
    return user
```

WHY WE DO THIS

A **dependency** is a function FastAPI runs *before* your endpoint. Any endpoint that lists `Depends(get_current_user)` is automatically protected.

`HTTPBearer(auto_error=False)` makes Swagger show the green **Authorize** button and lets us return **401** for a missing token (FastAPI's default would be 403).

The checks in order: token present -> signature valid -> not expired -> not revoked (blacklist) -> user still exists. Each failure gives its own clear message, matching the four cases the assessment lists: missing, invalid, expired, unauthorized.

Authorization rule: the endpoints never take an account id for 'me' or 'transactions'. The account always comes from the logged-in user in the token, so a user can never reach someone else's data.

Step 15: The endpoints: app/api/auth.py and app/api/accounts.py

Goal: Connect URLs to the services.

FILE: app/api/auth.py

```
from fastapi import APIRouter, Depends, status
from sqlalchemy.orm import Session

from app.api.deps import get_current_user, get_token_payload
from app.db.session import get_db
from app.models import User
from app.schemas.auth import (
    LoginRequest,
    MessageResponse,
    RegisterRequest,
    RegisterResponse,
    TokenResponse,
)
from app.services.auth_service import AuthService

router = APIRouter(prefix="/api/auth", tags=["Auth"])

@register.post("/register", response_model=RegisterResponse, status_code=status.HTTP_201_CREATED)
def register(data: RegisterRequest, db: Session = Depends(get_db)):
    user, account = AuthService(db).register(data)
    return RegisterResponse(
        id=user.id,
        name=user.name,
        email=user.email,
        account_number=account.account_number,
        created_at=user.created_at,
    )

@register.post("/login", response_model=TokenResponse)
def login(data: LoginRequest, db: Session = Depends(get_db)):
    token = AuthService(db).login(data)
    return TokenResponse(access_token=token)

@register.post("/logout", response_model=MessageResponse)
def logout(
    payload: dict = Depends(get_token_payload),
    _user: User = Depends(get_current_user),
    db: Session = Depends(get_db),
):
    AuthService(db).logout(payload)
    return MessageResponse(message="Logged out successfully")
```

FILE: app/api/accounts.py

```
from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session

from app.api.deps import get_current_user
from app.db.session import get_db
from app.models import User
from app.schemas.account import AccountResponse, AmountRequest, TransferRequest
from app.schemas.transaction import OperationResponse, TransactionResponse
from app.services.account_service import AccountService

router = APIRouter(prefix="/api/accounts", tags=["Accounts"])

@register.get("/me", response_model=AccountResponse)
def get_my_account(user: User = Depends(get_current_user), db: Session = Depends(get_db)):
    account = AccountService(db).get_account(user)
```

```

        return AccountResponse(
            account_id=account.id,
            account_number=account.account_number,
            account_holder_name=user.name,
            balance=account.balance,
            created_at=account.created_at,
        )

@router.post("/deposit", response_model=OperationResponse)
def deposit(
    data: AmountRequest,
    user: User = Depends(get_current_user),
    db: Session = Depends(get_db),
):
    account, tx = AccountService(db).deposit(user, data.amount)
    return OperationResponse(
        message="Deposit successful",
        balance=account.balance,
        transaction=TransactionResponse.model_validate(tx),
    )

@router.post("/withdraw", response_model=OperationResponse)
def withdraw(
    data: AmountRequest,
    user: User = Depends(get_current_user),
    db: Session = Depends(get_db),
):
    account, tx = AccountService(db).withdraw(user, data.amount)
    return OperationResponse(
        message="Withdrawal successful",
        balance=account.balance,
        transaction=TransactionResponse.model_validate(tx),
    )

@router.post("/transfer", response_model=OperationResponse)
def transfer(
    data: TransferRequest,
    user: User = Depends(get_current_user),
    db: Session = Depends(get_db),
):
    account, tx = AccountService(db).transfer(user, data.receiver_account_id, data.amount)
    return OperationResponse(
        message="Transfer successful",
        balance=account.balance,
        transaction=TransactionResponse.model_validate(tx),
    )

@router.get("/transactions", response_model=list[TransactionResponse])
def get_transactions(user: User = Depends(get_current_user), db: Session = Depends(get_db)):
    return AccountService(db).history(user)

```

WHY WE DO THIS

Look how short the route functions are: receive the validated input, call a service, return a response model. All the thinking is in the service layer. `response_model` filters the output so only the declared fields are returned and it documents the response shape in Swagger.

Step 16: main.py: put everything together and run the server

Goal: Create the app, register routers and error handlers, then start it.

FILE: app/main.py

```
import logging
from contextlib import asynccontextmanager

from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse

from app import models # noqa: F401 (registers the tables)
from app.api import accounts, auth
from app.core.exceptions import AppError
from app.db.base import Base
from app.db.session import engine

logger = logging.getLogger("bank")

@asynccontextmanager
async def lifespan(app: FastAPI):
    Base.metadata.create_all(bind=engine)
    yield

app = FastAPI(
    title="Basic Banking API",
    description="Register, log in, deposit, withdraw and transfer money.",
    version="1.0.0",
    lifespan=lifespan,
)

@app.exception_handler(AppError)
async def app_error_handler(request: Request, exc: AppError):
    headers = {"WWW-Authenticate": "Bearer"} if exc.status_code == 401 else None
    return JSONResponse(
        status_code=exc.status_code, content={"detail": exc.detail}, headers=headers
    )

@app.exception_handler(Exception)
async def unhandled_error_handler(request: Request, exc: Exception):
    logger.exception("Unhandled error") # full details only in the server log
    return JSONResponse(status_code=500, content={"detail": "Internal server error"})

app.include_router(auth.router)
app.include_router(accounts.router)
```

WHY WE DO THIS

The `lifespan` function runs once at startup and creates any missing tables. `@app.exception_handler(AppError)` converts our custom errors into JSON with the right status. The generic `Exception` handler is the safety net: the client only sees 'Internal server error' while the real traceback goes to the server log. The assessment says never expose implementation details in error responses.

```
uvicorn app.main:app --reload
```

CHECKPOINT - do not move on until this works

The terminal says Application startup complete. A file `bank.db` appears in your folder. Open <http://127.0.0.1:8000/docs> - you see all 8 endpoints grouped under Auth and Accounts. Leave the server running for the next step.

```
git add .
git commit -m "Schemas, repositories, services and API"
```

Step 17: Test it by hand in Swagger

Goal: Walk the whole happy path, then try to break it.

Happy path (follow exactly)

- **POST /api/auth/register** - click 'Try it out', use the sample body, Execute. Expect **201** and an **accountNumber**.
- Register a **second** user (different email) - you need someone to transfer to.
- **POST /api/auth/login** as user 1. Copy the value of **accessToken**.
- Click the green **Authorize** button at the top right, paste the token (only the token, no word 'Bearer'), Authorize, Close.
- **GET /api/accounts/me** - balance 0. Note the **accountId** of each user (log in as user 2 to see theirs).
- **POST /api/accounts/deposit** with `{"amount": 5000}` - balance 5000.
- **POST /api/accounts/withdraw** with 1000 - balance 4000.
- **POST /api/accounts/transfer** with user 2's id and amount 1500 - your balance 2500.
- **GET /api/accounts/transactions** - you see DEPOSIT, WITHDRAW, TRANSFER_DEBIT.
- **POST /api/auth/logout**, then call GET /me again with the same token - expect **401**.

Now try to break it: every edge case from the assessment

#	Edge case	What to do	Expect
1	Duplicate email	Register the same email again	409
2	Invalid login	Login with a wrong password / unknown email	401, same message for both
3-4	Negative / zero deposit	amount: -100 and amount: 0	422
5-6	Negative / zero withdrawal	amount: -5 and amount: 0	422
7	Insufficient balance	Withdraw or transfer more than you have	400, balance unchanged
8	Invalid JWT	Authorize with the text abc	401
9	Expired JWT	Set ACCESS_TOKEN_EXPIRE_MINUTES=-1 in .env, restart the server, login, call /me. Set it back to 30 afterwards	401 'Token has expired'
10	Non-existent account	Covered by the token check: a token for a deleted user is rejected	401
11	Non-existent receiver	receiverAccountId: 9999	404
12	Transfer to self	Use your own accountId	400
13	Database failure during transfer	Proven by an automated test (Step 18)	Balances unchanged
14	Concurrent transactions	Row locks + ordered locking + CHECK constraint (Steps 9, 12, 13)	No negative balance
-	No token at all	Click Authorize -> Logout, then call /me	401
-	Weak password	password: 'abc'	422
-	Blank name	name: ''	422

Stop the server with **Ctrl + C** when you are done, then delete `bank.db` so you start clean (it is git-ignored anyway).

Step 18: Automated tests

Goal: Write tests so a machine can check everything in seconds. The assessment requires them.

WHY WE DO THIS

A test is code that calls your API and checks the result. Once written, you can re-run all of them after any change to be sure you did not break something. The tests use an **in-memory database** that is created and destroyed for every test, so they never touch your real data.

conftest.py holds shared setup (fixtures) that pytest gives to every test:

FILE: tests/conftest.py

```
import os

# Must be set before the app is imported
os.environ.setdefault("SECRET_KEY", "test-secret-key-that-is-long-enough-for-hs256-signing")

import pytest
from fastapi.testclient import TestClient
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from sqlalchemy.pool import StaticPool

from app.db.base import Base
from app.db.session import get_db
from app.main import app

engine = create_engine(
    "sqlite://",
    connect_args={"check_same_thread": False},
    poolclass=StaticPool,
)
TestingSessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False)

@pytest.fixture()
def db_session():
    Base.metadata.create_all(bind=engine)
    session = TestingSessionLocal()
    try:
        yield session
    finally:
        session.close()
        Base.metadata.drop_all(bind=engine)

@pytest.fixture()
def client(db_session):
    def override_get_db():
        session = TestingSessionLocal()
        try:
            yield session
        finally:
            session.close()

    app.dependency_overrides[get_db] = override_get_db
    yield TestClient(app)
    app.dependency_overrides.clear()

def register_and_login(client, name="John Doe", email="john@example.com", password="Password123"):
    reg = client.post(
        "/api/auth/register", json={"name": name, "email": email, "password": password}
    )
    assert reg.status_code == 201, reg.text
    login = client.post("/api/auth/login", json={"email": email, "password": password})
    assert login.status_code == 200, login.text
    token = login.json()["accessToken"]
    return {"Authorization": f"Bearer {token}"}, reg.json()
```



```

@pytest.fixture()
def auth(client):
    headers, info = register_and_login(client)
    return headers

@pytest.fixture()
def second_user(client):
    return register_and_login(client, "Jane Roe", "jane@example.com", "Password456")

```

WATCH OUT

The line setting SECRET_KEY must stay **above** the imports from app. Settings are read the moment the app is imported, and the test environment has no .env value yet.

FILE: tests/test_auth.py

```

import time

import jwt
import pytest

from app.core.config import settings

def payload(**overrides):
    data = {"name": "John Doe", "email": "john@example.com", "password": "Password123"}
    data.update(overrides)
    return data

def test_register_success(client):
    res = client.post("/api/auth/register", json=payload())
    assert res.status_code == 201
    body = res.json()
    assert body["email"] == "john@example.com"
    assert len(body["accountNumber"]) == 10
    assert "password" not in body and "passwordHash" not in body

def test_register_duplicate_email(client):
    client.post("/api/auth/register", json=payload())
    res = client.post("/api/auth/register", json=payload(email="JOHN@example.com"))
    assert res.status_code == 409

@pytest.mark.parametrize(
    "bad",
    [
        {"email": "not-an-email"},
        {"name": " "},
        {"password": "short1A"},
        {"password": "alllowercase1"},
        {"password": "NOLOWERCASE1"},
        {"password": "NoDigitsHere"},
    ],
)
def test_register_validation(client, bad):
    res = client.post("/api/auth/register", json=payload(**bad))
    assert res.status_code == 422

def test_login_success(client):
    client.post("/api/auth/register", json=payload())
    res = client.post(
        "/api/auth/login", json={"email": "john@example.com", "password": "Password123"}
    )
    assert res.status_code == 200
    assert res.json()["tokenType"] == "bearer"
    assert res.json()["accessToken"]

```

```

def test_login_wrong_password(client):
    client.post("/api/auth/register", json=payload())
    res = client.post(
        "/api/auth/login", json={"email": "john@example.com", "password": "WrongPass1"}
    )
    assert res.status_code == 401

def test_login_unknown_email(client):
    res = client.post(
        "/api/auth/login", json={"email": "nobody@example.com", "password": "Password123"}
    )
    assert res.status_code == 401
    assert res.json()["detail"] == "Invalid email or password"

def test_missing_token_is_401(client):
    assert client.get("/api/accounts/me").status_code == 401

def test_invalid_token_is_401(client):
    res = client.get("/api/accounts/me", headers={"Authorization": "Bearer garbage"})
    assert res.status_code == 401

def test_expired_token_is_401(client, auth):
    expired = jwt.encode(
        {"sub": "1", "jti": "abc", "exp": int(time.time()) - 60},
        settings.secret_key,
        algorithm=settings.algorithm,
    )
    res = client.get("/api/accounts/me", headers={"Authorization": f"Bearer {expired}"})
    assert res.status_code == 401
    assert "expired" in res.json()["detail"].lower()

def test_logout_revokes_token(client, auth):
    assert client.get("/api/accounts/me", headers=auth).status_code == 200
    assert client.post("/api/auth/logout", headers=auth).status_code == 200
    assert client.get("/api/accounts/me", headers=auth).status_code == 401

def test_logout_requires_auth(client):
    assert client.post("/api/auth/logout").status_code == 401

```

FILE: tests/test_accounts.py

```

import pytest

from app.repositories.transaction_repository import TransactionRepository

def balance(client, headers):
    return client.get("/api/accounts/me", headers=headers).json()["balance"]

def test_get_my_account(client, auth):
    res = client.get("/api/accounts/me", headers=auth)
    assert res.status_code == 200
    body = res.json()
    assert body["accountHolderName"] == "John Doe"
    assert body["balance"] == 0
    assert {"accountId", "accountNumber", "createdAt"} <= body.keys()

def test_deposit_success(client, auth):
    res = client.post("/api/accounts/deposit", json={"amount": 5000}, headers=auth)
    assert res.status_code == 200
    assert res.json()["balance"] == 5000
    assert res.json()["transaction"]["type"] == "DEPOSIT"
    assert balance(client, auth) == 5000

```

```

@pytest.mark.parametrize("amount", [0, -10, "abc", 10.999])
def test_deposit_invalid_amount(client, auth, amount):
    res = client.post("/api/accounts/deposit", json={"amount": amount}, headers=auth)
    assert res.status_code == 422
    assert balance(client, auth) == 0

def test_deposit_requires_auth(client):
    assert client.post("/api/accounts/deposit", json={"amount": 10}).status_code == 401

def test_withdraw_success(client, auth):
    client.post("/api/accounts/deposit", json={"amount": 5000}, headers=auth)
    res = client.post("/api/accounts/withdraw", json={"amount": 2000}, headers=auth)
    assert res.status_code == 200
    assert res.json()["balance"] == 3000

@pytest.mark.parametrize("amount", [0, -5])
def test_withdraw_invalid_amount(client, auth, amount):
    res = client.post("/api/accounts/withdraw", json={"amount": amount}, headers=auth)
    assert res.status_code == 422

def test_withdraw_insufficient_balance(client, auth):
    client.post("/api/accounts/deposit", json={"amount": 100}, headers=auth)
    res = client.post("/api/accounts/withdraw", json={"amount": 100.01}, headers=auth)
    assert res.status_code == 400
    assert res.json()["detail"] == "Insufficient balance"
    assert balance(client, auth) == 100

def test_transfer_success(client, auth, second_user):
    jane_headers, jane_info = second_user
    jane_id = client.get("/api/accounts/me", headers=jane_headers).json()["accountId"]
    client.post("/api/accounts/deposit", json={"amount": 5000}, headers=auth)

    res = client.post(
        "/api/accounts/transfer",
        json={"receiverAccountId": jane_id, "amount": 1500},
        headers=auth,
    )
    assert res.status_code == 200
    assert balance(client, auth) == 3500
    assert balance(client, jane_headers) == 1500

    jane_txs = client.get("/api/accounts/transactions", headers=jane_headers).json()
    assert [t["type"] for t in jane_txs] == ["TRANSFER_CREDIT"]

def test_transfer_to_self(client, auth):
    my_id = client.get("/api/accounts/me", headers=auth).json()["accountId"]
    client.post("/api/accounts/deposit", json={"amount": 100}, headers=auth)
    res = client.post(
        "/api/accounts/transfer",
        json={"receiverAccountId": my_id, "amount": 10},
        headers=auth,
    )
    assert res.status_code == 400

def test_transfer_unknown_receiver(client, auth):
    client.post("/api/accounts/deposit", json={"amount": 100}, headers=auth)
    res = client.post(
        "/api/accounts/transfer",
        json={"receiverAccountId": 9999, "amount": 10},
        headers=auth,
    )
    assert res.status_code == 404
    assert balance(client, auth) == 100

@pytest.mark.parametrize("receiver", [0, -1, "abc"])
def test_transfer_invalid_receiver_id(client, auth, receiver):
    res = client.post(

```

```

        "/api/accounts/transfer",
        json={"receiverAccountId": receiver, "amount": 10},
        headers=auth,
    )
    assert res.status_code == 422

def test_transfer_insufficient_balance(client, auth, second_user):
    jane_headers, _ = second_user
    jane_id = client.get("/api/accounts/me", headers=jane_headers).json()["accountId"]
    res = client.post(
        "/api/accounts/transfer",
        json={"receiverAccountId": jane_id, "amount": 10},
        headers=auth,
    )
    assert res.status_code == 400
    assert balance(client, jane_headers) == 0

def test_transfer_is_atomic_when_database_fails(client, auth, second_user, monkeypatch):
    jane_headers, _ = second_user
    jane_id = client.get("/api/accounts/me", headers=jane_headers).json()["accountId"]
    client.post("/api/accounts/deposit", json={"amount": 5000}, headers=auth)

    original = TransactionRepository.create
    calls = {"n": 0}

    def flaky_create(self, *args, **kwargs):
        calls["n"] += 1
        if calls["n"] == 2: # fail on the second row of the transfer
            raise RuntimeError("simulated database failure")
        return original(self, *args, **kwargs)

    monkeypatch.setattr(TransactionRepository, "create", flaky_create)

    with pytest.raises(RuntimeError):
        client.post(
            "/api/accounts/transfer",
            json={"receiverAccountId": jane_id, "amount": 1500},
            headers=auth,
        )

    monkeypatch.undo()
    assert balance(client, auth) == 5000
    assert balance(client, jane_headers) == 0
    history = client.get("/api/accounts/transactions", headers=auth).json()
    assert [t["type"] for t in history] == ["DEPOSIT"]

def test_transactions_only_show_own(client, auth, second_user):
    jane_headers, _ = second_user
    client.post("/api/accounts/deposit", json={"amount": 5000}, headers=auth)
    client.post("/api/accounts/withdraw", json={"amount": 1000}, headers=auth)
    client.post("/api/accounts/deposit", json={"amount": 7}, headers=jane_headers)

    mine = client.get("/api/accounts/transactions", headers=auth).json()
    hers = client.get("/api/accounts/transactions", headers=jane_headers).json()
    assert [(t["type"], t["amount"]) for t in mine] == [("DEPOSIT", 5000), ("WITHDRAW", 1000)]
    assert [(t["type"], t["amount"]) for t in hers] == [("DEPOSIT", 7)]
    assert {"id", "type", "amount", "status", "createdAt"} <= mine[0].keys()

def test_transactions_requires_auth(client):
    assert client.get("/api/accounts/transactions").status_code == 401

```

```
python -m pytest -v
```

CHECKPOINT - do not move on until this works

Everything is green: 37 passed (a single yellow deprecation warning from the test client library is normal). Read the test names - they map directly to the assessment's test areas: registration, login, deposit, withdrawal, transfer, insufficient balance and unauthorized request.

The **atomicity test** deserves a second look. It replaces the function that writes history rows so it crashes on its 2nd call, then does a transfer and finally asserts that **both balances and both histories are exactly as before**. If you remove the `rollback()` in the service, this test fails - try it, then put it back.

```
git add .
git commit -m "Add tests"
```

Step 19: README.md

Goal: Document the project. The assessment requires 13 specific sections.

Create `README.md` in the project root. Write it in your own words where you can - reviewers read this first. Replace `<your-repository-url>` with your real URL later.

FILE: README.md

```
# Basic Banking API

## 1. Project overview
A REST API for a basic banking application. Users can register, log in, view their
account, deposit, withdraw, transfer money to another account, see their transaction
history and log out. Every protected endpoint needs a JWT.

## 2. Technology stack
- Python 3.11+
- FastAPI (web framework, generates Swagger docs)
- SQLAlchemy 2.x (ORM) with SQLite (PostgreSQL / MySQL work by changing `DATABASE_URL`)
- Pydantic v2 (request / response validation)
- PyJWT (tokens) and bcrypt (password hashing)
- pytest + httpx (tests)

## 3. Prerequisites
- Python 3.11 or newer
- Git

## 4. Installation instructions
```bash
git clone <your-repository-url>
cd <repository-folder>
python -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate
pip install -r requirements.txt
```

## 5. Environment variables
Copy the sample file and set a real secret:
```bash
cp .env.example .env # Windows: copy .env.example .env
```

Variable	Meaning
DATABASE_URL	Database connection string (default `sqlite:///./bank.db`)
SECRET_KEY	Long random string used to sign JWTs (required)
ALGORITHM	JWT signing algorithm (default `HS256`)
ACCESS_TOKEN_EXPIRE_MINUTES	Token lifetime in minutes (default `30`)

Generate a key with: `python -c "import secrets; print(secrets.token_hex(32))"`

## 6. Database setup
Nothing to run manually. Tables are created automatically when the app starts.
With SQLite the file `bank.db` appears in the project folder on first start.

## 7. How to run the application
```bash
uvicorn app.main:app --reload
```

The API runs at http://127.0.0.1:8000

## 8. API documentation
Swagger UI: http://127.0.0.1:8000/docs (ReDoc: /redoc).
```

To test protected endpoints: call `POST /api/auth/login`, copy `accessToken`, click **Authorize** in Swagger and paste the token.

| Method | Path | Auth | Description |
|--------|----------------------------|------|---|
| POST | /api/auth/register | No | Register a user (an account is created automatically) |
| POST | /api/auth/login | No | Get a JWT access token |
| POST | /api/auth/logout | Yes | Revoke the current token |
| GET | /api/accounts/me | Yes | Current user's account details |
| POST | /api/accounts/deposit | Yes | Deposit money |
| POST | /api/accounts/withdraw | Yes | Withdraw money |
| POST | /api/accounts/transfer | Yes | Transfer to another account id |
| GET | /api/accounts/transactions | Yes | Current user's transaction history |

9. Authentication approach (including logout strategy)

- Passwords are hashed with bcrypt. Plain passwords are never stored or logged.
- Login returns a signed JWT (`sub` = user id, `exp`, and a unique `jti`) that is sent as `Authorization: Bearer <token>`.
- Missing, invalid, expired or revoked tokens return `401 Unauthorized`.
- **Logout strategy: token blacklist.** JWTs are stateless, so on logout the token's `jti` is stored in the `blacklisted\_tokens` table. Every request checks that table, so a logged-out token stops working immediately. Rows are deleted once the token would have expired anyway. Access tokens are also short lived (30 minutes by default).

10. Database design

- `users`: id, name, email (unique), password_hash, created_at
- `accounts`: id, user_id (unique, FK), account_number (unique), balance, created_at
- `transactions`: id, account_id (FK), type, amount, status, reference_id, created_at
- `blacklisted\_tokens`: id, jti (unique), expires_at

One user has one account. Money uses `Numeric(12, 2)` (never float) and a database `CHECK (balance >= 0)` guards against negative balances.

11. Design decisions

- **Layers:** api (HTTP) -> services (business rules) -> repositories (database queries).
- **Atomic transfers:** debit, credit and both transaction rows happen in one database transaction with a single `commit()`. Any error triggers `rollback()`.
- **Concurrency:** account rows are locked with `SELECT ... FOR UPDATE` and, in a transfer, always in ascending id order so two opposite transfers cannot deadlock.
- **Transfers create two history rows** (`TRANSFER\_DEBIT` for sender, `TRANSFER\_CREDIT` for receiver) that share a `reference\_id`, so each user only sees their own side.
- **Errors:** 400 business rule broken, 401 auth problem, 404 not found, 409 duplicate email, 422 validation, 500 generic message (details only go to the server log).
- **Login errors are generic** ("Invalid email or password") so attackers cannot discover which emails are registered.

12. How to run tests

```
``bash
python -m pytest -v
````
```

Tests use an in-memory SQLite database, so your real data is never touched.

### ## 13. Known limitations

- SQLite ignores `FOR UPDATE`; it serialises writes instead. Use PostgreSQL for true row-level locking under heavy concurrency.
- Tables are created with `create\_all`, not migrations.
- No refresh tokens, rate limiting or pagination.
- One account per user.

| #     | Required section                      | Where in the file above |
|-------|---------------------------------------|-------------------------|
| 1-3   | Overview, stack, prerequisites        | Sections 1-3            |
| 4-7   | Install, env variables, DB setup, run | Sections 4-7            |
| 8     | API documentation                     | Section 8               |
| 9     | Authentication incl. logout strategy  | Section 9               |
| 10-11 | Database design, design decisions     | Sections 10-11          |
| 12-13 | How to run tests, known limitations   | Sections 12-13          |

## Step 20: Push to GitHub and do the final check

**Goal:** Publish the repository and prove a stranger can run it.

- Create a new **empty public** repository on github.com (no README, no .gitignore - you already have them).

```
git branch -M main
git remote add origin https://github.com/<your-username>/<your-repo>.git
git add .
git commit -m "Final version"
git push -u origin main
```

### The 'fresh clone' test (the assessment asks for this)

Pretend you are the reviewer. In a **different** folder, follow only your own README:

macOS / Linux

```
cd ..
git clone https://github.com/<your-username>/<your-repo>.git fresh-test
cd fresh-test
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
python -m pytest -q
uvicorn app.main:app
```

Windows (PowerShell)

```
cd ..
git clone https://github.com/<your-username>/<your-repo>.git fresh-test
cd fresh-test
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env
python -m pytest -q
uvicorn app.main:app
```

#### WATCH OUT

.env.example contains a placeholder SECRET\_KEY, so the fresh clone starts. In a real project you would replace it with a real random value. If the clone fails, your README is missing a step: fix the README, not just your own machine.

### Submission checklist

| Done?                    | Check                                                                             |
|--------------------------|-----------------------------------------------------------------------------------|
| <input type="checkbox"/> | Repository is <b>public</b> (open the link in a private/incognito browser window) |
| <input type="checkbox"/> | Contains: app/, tests/, requirements.txt, README.md, .env.example, .gitignore     |
| <input type="checkbox"/> | Does NOT contain: venv/, .env, __pycache__/, bank.db                              |
| <input type="checkbox"/> | All 8 endpoints work in Swagger at /docs (with the Authorize button)              |
| <input type="checkbox"/> | python -m pytest shows all tests passing on the fresh clone                       |
| <input type="checkbox"/> | README has all 13 sections, including the logout strategy                         |
| <input type="checkbox"/> | No real secrets anywhere in the code or in the Git history                        |
| <input type="checkbox"/> | You can explain every file in your own words (see the questions at the end)       |

## Appendix A: Requirement checklist

How each mandatory item in the assessment is satisfied.

| Assessment section    | Requirement                                                                          | Where it is done                                       |
|-----------------------|--------------------------------------------------------------------------------------|--------------------------------------------------------|
| 1-2 API summary       | 8 endpoints                                                                          | api/auth.py, api/accounts.py                           |
| 3 Registration        | Validation, hashed password, duplicate email                                         | schemas/auth.py, services/auth_service.py              |
| 4 Login               | JWT access token, bearer type                                                        | core/security.py, services/auth_service.py             |
| 5 Account details     | Only own account                                                                     | api/accounts.py (account comes from token user)        |
| 6 Deposit             | amount > 0, balance up, record transaction                                           | schemas/account.py, services/account_service.py        |
| 7 Withdraw            | amount > 0, not above balance, record                                                | services/account_service.py                            |
| 8 Transfer            | No self transfer, both sides updated, ATOMIC                                         | services/account_service.py (single commit + rollback) |
| 9 Transaction history | Own transactions only                                                                | repositories/transaction_repository.py                 |
| 10 Logout             | Strategy explained in README                                                         | Token blacklist (models/blacklisted_token.py)          |
| 11 Database           | Relational DB, entities                                                              | SQLite + models/                                       |
| 12 Authentication     | Bearer header; missing/invalid/expired                                               | api/deps.py                                            |
| 13 Validation         | Pydantic models                                                                      | schemas/                                               |
| 14 Error handling     | 400/401/404/409/422, no internals leaked                                             | core/exceptions.py, main.py                            |
| 15 Security           | Hashed passwords, no secrets in repo/errors                                          | core/security.py, .gitignore, main.py                  |
| 16 Project structure  | Clean layers                                                                         | Whole app/ folder                                      |
| 17 API documentation  | Swagger incl. protected endpoints                                                    | /docs with Authorize (HTTPBearer)                      |
| 18 Testing            | Registration, login, deposit, withdraw, transfer, insufficient balance, unauthorized | tests/                                                 |
| 19 Edge cases         | All 14                                                                               | Step 17 table                                          |
| 20 README             | 13 sections                                                                          | README.md                                              |
| 22 Submission         | Repo contents / exclusions                                                           | Step 20 checklist                                      |

## Appendix B: If something goes wrong

| Symptom                                           | Likely cause and fix                                                                    |
|---------------------------------------------------|-----------------------------------------------------------------------------------------|
| 'secret_key Field required'                       | .env missing or SECRET_KEY empty. Re-do Step 5                                          |
| 'No module named app' / 'No module named fastapi' | Wrong folder (run commands from the project root) or venv not activated                 |
| 'email-validator is not installed'                | Install with: pip install "pydantic[email]" (it is in requirements.txt)                 |
| Swagger shows 401 on protected endpoints          | You did not click Authorize, or pasted the token with 'Bearer ' in front, or logged out |
| 'Address already in use'                          | Another server is still running. Stop it with Ctrl + C or use --port 8001               |
| Table has no column ... after changing a model    | SQLite does not update existing tables. Stop the server, delete bank.db, start again    |
| Tests fail with the settings error                | The SECRET_KEY line in tests/conftest.py must come before the app imports               |
| ImportError: circular import                      | Check that models import from app.db.base and never from services or api                |
| Windows: 'running scripts is disabled'            | Set-ExecutionPolicy -Scope CurrentUser RemoteSigned                                     |

## Appendix C: Questions a reviewer may ask you

Practice answering these out loud. If you can answer them, the code is truly yours.



**1. Why do we hash passwords instead of encrypting them?**

Hashing is one-way, so even the developers and a database thief cannot recover the password. Encryption can be reversed.

**2. What is inside a JWT and who can read it?**

sub, jti, iat, exp. Anyone can read it (it is only base64), but nobody can change it without the secret key because of the signature.

**3. JWTs are stateless. How does your logout work?**

Token blacklist: the jti is saved in a table and checked on every request. Trade-off: one extra database lookup per request.

**4. What does atomic mean in your transfer?**

All steps succeed together or none do. One commit at the end, rollback on any exception.

**5. What happens if two transfers hit the same account at the same time?**

Row locks with FOR UPDATE make the second wait; accounts are locked in id order to avoid deadlock; the CHECK constraint is a final guard.

**6. Why Decimal/Numeric and not float for money?**

Floats cannot represent most decimals exactly and cause rounding errors.

**7. Why do login errors say 'Invalid email or password' for both cases?**

So attackers cannot find out which emails have accounts.

**8. Why separate routes, services and repositories?**

Each layer has one job, so code is easier to read, test and change (for example, swap the database without touching business rules).

**9. Why can't a user see another user's account?**

Endpoints never accept an account id for reading; the account is looked up from the user id inside the verified token.

**10. Why is .env not in Git but .env.example is?**

The real secret would let anyone forge tokens. The example shows which variables are needed without any real values.